Smart Programming : YouTube Channel

An investment in Knowledge pays the best interest….

JDBC (PART 5)

Steps to develop JDBC Application :

1. Load & Register driver

- Sun microsystem created an interface i.e. Driver but its implementations are provided by database vendors (MySQL, Oracle)
- Database vendors implemented Driver interface and provides this implementation in the form of jar file which we can download from their respective websites
  - for ex MySQL provided mysql-connector.jar file
  - for ex Oracle provides ojdbc-no.jar file
- We can download these jar files and embed them into our project or say we have to set the path of these jar files.
- Then we have to load the Driver class in JVM
  - Class.forName("package name with Driver class name");
- After this code JVM will search for this package and Driver class in all three locations (1. current

- project 2. in ext folder 3. in classpath environment variables)
    - If Driver class is not found then it will provide exception "ClassNotFoundException"
    - If Driver class is found it will load the bytecode in JVM
- JVM will register the Driver
    - (DriverManager.registerDriver())

---

2. Establish connection b/w java application and database

- Connection con=
  DriverManager.getConnection(-,-,-);
- Due to above code a connection is set between Java Application and database

- In java we cannot create an object of interface then how Connection interface object is created
  - DriverManager.getConnection() method is not creating object for Connection interface, it creates an object of connection provided by database vendors

---

3. Create statement object

- Statement st=con.createStatement();

---

4. Send and execute SQL query

- ResultSet rs=st.executeQuery("select * from register");

- int i=st.executeUpdate("insert into register values('-','-','-')");

---

5. Process the result (ResultSet or process int value)

- while(rs.next())

```
{
        System.out.print(rs.getString(1));
        System.out.print(", "+rs.getString(2));
        System.out.print(", "+rs.getString(3));
}
```

- if(i>0)

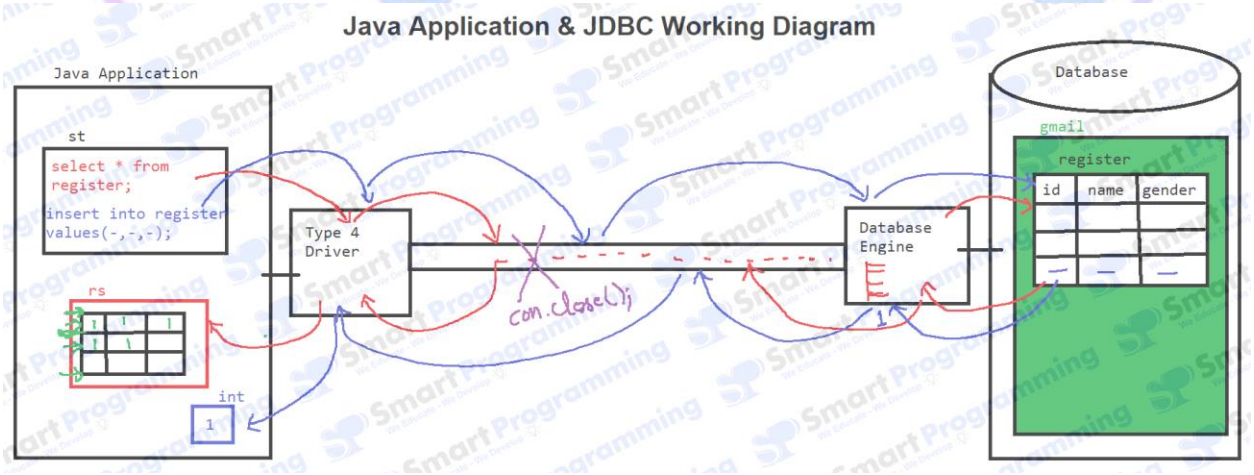
```
{
    System.out.println("values inserted
                                successfully");
}
else
{
    System.out.println("values insertion failed");
}
```

Smart Programming
We Educate
We Develop

6. Close all the resources (connection, statement, resultset)

- st.close();
- rs.close();
- con.close();

Diagrams :-



Model Architecture of Classes & Interfaces in Java, MySQL & Oracle

SUN MICROSYSTEMS (Java)

```
package java.sql;
interface Driver
{
}
class DriverManager
{
  void registeDriver(Driver driver)
  {
     //code
  }
  Connection getCOnnection(-,-,-)
  {
     Connection con=new OracleConnection();
     return con;
  }
}
interface Connection
{
  Statement createStatement()
  {
  }
}
----
many more classes & interfaces...
```

MySQL (jar file)

```
public class Driver implements java.sql.Driver
{
      static
      {
         DriverManager.registerDriver(driver ref variable);
      }
}
class MySqlConnection implement java.sql.Connection
{
}
----
many more classes & interfaces...
```

ORACLE (jar files)

```
public class OracleDriver extends
                    oracle.jdbc.driver.OracleDriver
{
}
public class OracleDriver implements java.sql.Driver
{
   static
   {
      DriverManager.registerDriver(driver ref vr);
   }
}
class OracleConnection implement java.sql.Connection
{
}
----
many more classes & interfaces...
```

JdbcTesting.java

```
public class JdbcTesting
{
    public static void main(String[] args) throws Exception
    {
        Class.forName("com.mysql.jdbc.Driver"); //MySQL
        //Class.forName("oracle.jdbc.OracleDriver");  //Oracle

        Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306/gmail","root","root");
        Statement st=con.createStatement();
//        ResultSet rs=st.executeQuery("select * from register");
//        while(rs.next())
//        {
//            System.out.print(rs.getString(1));
//            System.out.print(", "+rs.getString(2));
//            System.out.print(", "+rs.getString(3));
//        }

        int i=st.executeUpdate("insert into register values('-','-','-')");
        if(i>0)
        {
            System.out.println("values inserted successfully");
        }
        else
        {
            System.out.println("values insertion failed");
        }

        con.close();
    }
}
```

Company Links & Contacts

Company Name: Smart Programming (+91 62838-30308)

Address : Chandigarh & Mohali (Punjab), India

Websites: https://www.smartprogramming.in/

https://courses.smartprogramming.in

Android App:
https://play.google.com/store/apps/details?id=com.smartprogramming

YouTube Channel:
https://www.youtube.com/c/SmartProgramming